

Cara kerja push button

Fungsi pull-up dan pull-down resistor

Pembacaan input dengan polling

Sistem event-driven menggunakan gpiozero

Pemahaman ini menjadi fondasi penting sebelum memasuki pembahasan sensor dan sistem interaktif yang lebih kompleks.

BAB 7. Dasar Motor dan Kendali Gerak

Pada level Beginner, pembaca telah memahami bagaimana Raspberry Pi berinteraksi dengan dunia fisik melalui input dan output digital. LED menyala, buzzer berbunyi, dan tombol dapat dibaca. Namun, seluruh eksperimen tersebut masih bersifat statis.

Pada bab ini, kita memasuki tahap baru dalam robotika: **gerak (motion)**.

Gerak adalah elemen fundamental dalam sistem robot. Tanpa motor, robot tidak dapat berpindah posisi, menavigasi lingkungan, atau melakukan aksi fisik. Oleh karena itu, pemahaman tentang motor dan kendali gerak menjadi fondasi utama sebelum membangun robot mobile pada bab berikutnya.

7.1 Motor DC dan Prinsip Kerja

Motor DC (Direct Current Motor) adalah aktuator listrik yang mengubah energi listrik menjadi energi mekanik berupa putaran.

Prinsip Dasar Kerja Motor DC

Motor DC bekerja berdasarkan prinsip elektromagnetik:

1. Ketika arus listrik mengalir melalui kumparan di dalam motor,
2. Kumparan tersebut menghasilkan medan magnet,
3. Interaksi antara medan magnet kumparan dan magnet permanen di dalam motor menghasilkan gaya putar,
4. Poros motor berputar.

Arah putaran motor ditentukan oleh arah arus listrik yang mengalir. Jika polaritas dibalik, arah putaran juga akan berubah.

Karakteristik Motor DC pada Robot

Pada Trainer Kit EduPi Robo, motor DC biasanya digunakan untuk:

- Menggerakkan roda kiri dan kanan
- Menggerakkan mekanisme kecil
- Sistem aktuasi sederhana

Motor DC memiliki dua karakteristik penting:

- Tegangan menentukan kecepatan
- Polaritas menentukan arah

Namun, Raspberry Pi tidak dapat langsung mengendalikan motor DC, karena:

- Arus motor jauh lebih besar dibanding kemampuan GPIO
- Motor menghasilkan noise listrik (back EMF)

Karena itu diperlukan motor driver.

7.2 Motor Driver (L298N / Sejenis)

Motor driver adalah modul penguat daya yang memungkinkan Raspberry Pi mengendalikan motor secara aman. Salah satu motor driver yang umum digunakan adalah **L298N**, yang bekerja berdasarkan konsep **H-Bridge**.

Apa Itu H-Bridge?

H-Bridge adalah rangkaian yang memungkinkan arus mengalir dalam dua arah melalui motor.

Dengan mengatur kombinasi input HIGH dan LOW, kita dapat:

- Menggerakkan motor maju
- Menggerakkan motor mundur
- Menghentikan motor
- Mengatur kecepatan dengan PWM

Struktur Pin Umum L298N

Untuk satu motor:

- IN1
- IN2
- ENA (Enable / PWM)

Tabel logika dasar arah :

IN1	IN2	Arah Motor
HIGH	LOW	Maju
LOW	HIGH	Mundur
LOW	LOW	Stop
HIGH	HIGH	Brake

7.3 Kontrol Arah dan Kecepatan (PWM)

PWM (Pulse Width Modulation) adalah teknik untuk mengatur daya rata-rata dengan mengubah lebar pulsa sinyal digital.

Alih-alih memberikan tegangan analog, Raspberry Pi memberikan sinyal HIGH dan LOW dengan frekuensi tertentu.

Konsep Duty Cycle

Duty cycle adalah persentase waktu sinyal berada dalam kondisi HIGH.

- 0% → motor berhenti
- 50% → motor berputar sedang
- 100% → motor berputar penuh

Secara teknis:

Tegangan efektif = Duty Cycle $\times$ Tegangan Sumber

Eksperimen 1: Kontrol Arah Motor

Asumsi Koneksi:

- IN1 $\rightarrow$ GPIO 5
- IN2 $\rightarrow$ GPIO 6
- ENA $\rightarrow$ GPIO 13 (PWM)

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)

#SETUP GPIO

IN1 = 5
IN2 = 6
ENA = 13

GPIO.setup(IN1, GPIO.OUT)
GPIO.setup(IN2, GPIO.OUT)
GPIO.setup(ENA, GPIO.OUT)

pwm = GPIO.PWM(ENA, 1000)
pwm.start(0)
```

```

def motor_maju(speed):
    GPIO.output(IN1, GPIO.HIGH)
    GPIO.output(IN2, GPIO.LOW)
    pwm.ChangeDutyCycle(speed)

def motor_mundur(speed):
    GPIO.output(IN1, GPIO.LOW)
    GPIO.output(IN2, GPIO.HIGH)
    pwm.ChangeDutyCycle(speed)

def motor_stop():
    pwm.ChangeDutyCycle(0)
try:
    print("Motor maju")
    motor_maju(60)
    time.sleep(3)
    print("Motor mundur")
    motor_mundur(60)
    time.sleep(3)
    print("Motor stop")
    motor_stop()

```

```
        time.sleep(2)
except KeyboardInterrupt:
    pass
finally:
    pwm.stop()
    GPIO.cleanup()
```

Indikator Keberhasilan

- Motor berputar maju selama 3 detik
- Motor berputar mundur selama 3 detik
- Motor berhenti tanpa error

7.4 Kalibrasi Motor Kiri & Kanan

Dalam praktik nyata, dua motor DC **tidak pernah identik**. Meskipun diberi duty cycle yang sama, kecepatannya bisa berbeda.

Akibatnya:

- Robot cenderung belok ke kiri atau kanan
- Jalur tidak lurus
- Navigasi menjadi tidak akurat

Mengapa Terjadi Perbedaan?

Beberapa faktor penyebab:

- Perbedaan toleransi manufaktur
- Gesekan roda
- Distribusi berat robot
- Tegangan baterai tidak stabil

Metode Kalibrasi Sederhana

Langkah umum:

1. Jalankan kedua motor dengan duty cycle sama (misal 60%)
2. Amati arah gerak robot
3. Jika condong ke kanan → kurangi kecepatan motor kiri
4. Jika condong ke kiri → kurangi kecepatan motor kanan
5. Ulangi hingga robot bergerak lurus

Contoh Kalibrasi Software

```
LEFT_SPEED = 60
```

```
RIGHT_SPEED = 55 # dikurangi karena motor kanan lebih cepat
```

```
def maju():
```

```
    pwm_left.ChangeDutyCycle(LEFT_SPEED)
```

```
    pwm_right.ChangeDutyCycle(RIGHT_SPEED)
```

* Pendekatan ini disebut **software compensation**.

Pendekatan Lebih Lanjut (Level Advanced Preview)

Pada sistem profesional, kalibrasi dilakukan menggunakan:

- Encoder roda
- Closed-loop control
- PID Controller

Namun pada level ini, kita masih menggunakan metode open-loop berbasis pengamatan.

Refleksi Pembelajaran

Pada bab ini, pembaca telah memahami:

- Prinsip kerja motor DC
- Peran motor driver (H-Bridge)
- Konsep PWM untuk kontrol kecepatan
- Pentingnya kalibrasi motor kiri dan kanan

Pemahaman ini menjadi fondasi utama untuk membangun **robot mobile berbasis differential drive** pada bab berikutnya.

Penutup Bab

Motor adalah jantung dari sistem robot bergerak. Tanpa kendali motor yang baik, sistem navigasi, sensor, bahkan AI tidak akan memberikan hasil optimal.

Pada bab berikutnya, kita akan menggabungkan dua motor menjadi sistem **robot mobile dasar**, mempelajari konsep differential drive, serta memahami bagaimana baris kode diterjemahkan menjadi perilaku nyata di dunia fisik.

BAB 8. Sensor Untuk Navigasi

Pada bab sebelumnya, pembaca telah memahami bagaimana mengendalikan motor DC serta melakukan kalibrasi gerak robot. Namun, robot yang hanya bisa bergerak maju–mundur tanpa memahami lingkungan sekitarnya masih bersifat “buta”.

Agar robot dapat bergerak secara cerdas, ia membutuhkan **sensor navigasi**.

Sensor memungkinkan robot:

- Mendeteksi keberadaan objek
- Mengukur jarak
- Menghindari tabrakan
- Mengambil keputusan berdasarkan kondisi lingkungan

Pada bab ini, kita akan mempelajari penggunaan **sensor ultrasonik** untuk navigasi dasar serta membangun logika obstacle avoidance sederhana menggunakan konsep state machine.

8.1 Sensor Ultrasonik untuk Obstacle Detection

Sensor ultrasonik yang umum digunakan pada robot edukasi adalah HC-SR04.

Prinsip Kerja Sensor Ultrasonik

Sensor ultrasonik bekerja dengan prinsip **time of flight**:

1. Sensor mengirimkan gelombang suara frekuensi tinggi (ultrasonik).
2. Gelombang tersebut memantul ketika mengenai objek.
3. Sensor menerima pantulan (echo).
4. Jarak dihitung berdasarkan waktu tempuh gelombang.

Rumus Perhitungan Jarak

Jarak = 2 Waktu × Kecepatan Suara

Kecepatan suara ≈ 343 m/s.

Jarak (cm) = 0.5 x waktu (μ s) x 0.0343

Pembagian dua dilakukan karena gelombang menempuh jarak pergi dan kembali.

Konfigurasi Pin HC-SR04

- VCC → 5V
- GND → GND
- TRIG → GPIO Output
- ECHO → GPIO Input

⚠ Catatan Penting

Pin ECHO mengeluarkan sinyal 5V. Karena Raspberry Pi hanya mendukung 3.3V, diperlukan **voltage divider** untuk menurunkan tegangan agar aman.

8.2 Eksperimen 1: Membaca Jarak Sensor Ultrasonik

Tujuan Eksperimen

- Membaca jarak objek di depan robot
- Memahami cara kerja time-based measurement
- Menguji kestabilan pembacaan sensor

Asumsi Wiring

- TRIG → GPIO 23
- ECHO → GPIO 24 (melalui voltage divider)

Kode Program Pembacaan Jarak.

```
import RPi.GPIO as GPIO

import time

GPIO.setmode(GPIO.BCM)

TRIG = 23

ECHO = 24

GPIO.setup(TRIG, GPIO.OUT)

GPIO.setup(ECHO, GPIO.IN)

def baca_jarak():

    # Pastikan TRIG LOW

    GPIO.output(TRIG, False)

    time.sleep(0.05)
```

```

    # Kirim pulsa 10 mikrodetik
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)

    # Tunggu ECHO HIGH
    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

    # Tunggu ECHO LOW
    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150 # 34300 / 2
    distance = round(distance, 2)

    return distance

try:
    while True:
        jarak = baca_jarak()
        print(f"Jarak: {jarak} cm")
        time.sleep(1)
except KeyboardInterrupt:
    print("Program dihentikan")

```

```
finally:
```

```
    GPIO.cleanup()
```

Dokumentasi Program

- Inisialisasi GPIO
Program mengatur TRIG sebagai output dan ECHO sebagai input.
- Pengiriman Pulsa
TRIG diberi pulsa HIGH selama 10 mikrodetik untuk memicu pengiriman gelombang ultrasonik.
- Pengukuran Waktu
Program mengukur waktu antara ECHO HIGH dan ECHO LOW.
- Perhitungan Jarak
Waktu dikalikan 17150 untuk mendapatkan jarak dalam centimeter.
- Loop Monitoring
Sensor membaca jarak setiap 1 detik.

Indikator Keberhasilan

- Jarak berubah saat objek mendekat atau menjauh
- Pembacaan stabil (tidak melonjak ekstrem)
- Tidak muncul error timeout

8.3 Logika Obstacle Avoidance Sederhana

Setelah dapat membaca jarak, langkah berikutnya adalah membuat robot **mengambil keputusan otomatis**.

Konsep Dasar Avoidance

Jika:

- Jarak > 20 cm $\rightarrow$ robot maju
- Jarak ≤ 20 cm $\rightarrow$ robot berhenti dan belok

Ini disebut **rule-based decision system**.

8.4 Eksperimen 2: Robot Menghindari Halangan

Asumsi

- Motor kiri dan kanan sudah dikonfigurasi
- Sensor ultrasonik di depan robot

Kode Program Avoidance

```
import RPi.GPIO as GPIO

import time

GPIO.setmode(GPIO.BCM)

# Motor kiri

IN1_L = 5

IN2_L = 6
```

```
ENA_L = 13
```

```
# Motor kanan
```

```
IN1_R = 16
```

```
IN2_R = 20
```

```
ENA_R = 21
```

```
# Ultrasonik
```

```
TRIG = 23
```

```
ECHO = 24
```

```
# Setup pin
```

```
for pin in [IN1_L, IN2_L, ENA_L, IN1_R, IN2_R, ENA_R,  
TRIG]:
```

```
    GPIO.setup(pin, GPIO.OUT)
```

```
GPIO.setup(ECHO, GPIO.IN)
```

```
pwm_left = GPIO.PWM(ENA_L, 1000)
```

```
pwm_right = GPIO.PWM(ENA_R, 1000)
```

```
pwm_left.start(60)

pwm_right.start(60)


def maju():

    GPIO.output(IN1_L, 1)

    GPIO.output(IN2_L, 0)

    GPIO.output(IN1_R, 1)

    GPIO.output(IN2_R, 0)


def belok_kanan():

    GPIO.output(IN1_L, 1)

    GPIO.output(IN2_L, 0)

    GPIO.output(IN1_R, 0)

    GPIO.output(IN2_R, 1)


def stop():

    GPIO.output(IN1_L, 0)

    GPIO.output(IN2_L, 0)

    GPIO.output(IN1_R, 0)

    GPIO.output(IN2_R, 0)
```

```

def baca_jarak():
    GPIO.output(TRIG, False)

    time.sleep(0.05)

    GPIO.output(TRIG, True)

    time.sleep(0.00001)

    GPIO.output(TRIG, False)

    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()

    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()

    duration = pulse_end - pulse_start

    distance = duration * 17150

    return round(distance, 2)

try:
    while True:
        jarak = baca_jarak()

```

```

        print("Jarak:", jarak)

    if jarak > 20:
        maju()
    else:
        stop()
        time.sleep(0.5)
        belok_kanan()
        time.sleep(0.7)

except KeyboardInterrupt:
    print("Program dihentikan")
finally:
    pwm_left.stop()
    pwm_right.stop()
    GPIO.cleanup()

```

Dokumentasi Sistem

1. Arsitektur Sistem

Sensor → Logika → Motor

2. Mekanisme Kerja

- Sensor membaca jarak
- Sistem membandingkan dengan threshold
- Jika aman → robot maju
- Jika tidak aman → robot berhenti dan belok

3. Karakteristik Sistem

- Bersifat open-loop
- Tidak menyimpan memori lingkungan
- Menggunakan rule-based decision

8.5 Konsep State Machine Dasar Robot

State machine adalah sistem yang bekerja berdasarkan kondisi (state) tertentu.

Contoh state robot:

- FORWARD
- STOP
- TURN

Transisi antar state dipicu oleh kondisi jarak.

Pendekatan ini membuat sistem lebih terstruktur dan siap dikembangkan ke level lanjut.

Refleksi Pembelajaran

Pada bab ini, pembaca telah memahami:

- Cara kerja sensor ultrasonik
- Teknik pengukuran jarak berbasis waktu

- Logika obstacle avoidance sederhana
- Konsep state machine dasar

Robot kini tidak lagi bergerak tanpa arah, tetapi mulai memiliki **kemampuan persepsi dan respons terhadap lingkungan**.

Penutup Bab

Sensor adalah indera robot. Tanpa sensor, robot tidak dapat beradaptasi.

BAB 9. Networking dengan Raspberry Pi

Pada level Beginner, pembaca telah memahami bagaimana Raspberry Pi berinteraksi dengan dunia fisik melalui GPIO, sensor, dan aktuator. Pada level Intermediate, sistem mulai berkembang menjadi robot bergerak.

Namun dalam dunia nyata, sistem modern tidak bekerja secara terisolasi.

Robot, sensor, dan perangkat embedded saat ini hampir selalu **terhubung ke jaringan**. Koneksi jaringan memungkinkan:

- Monitoring jarak jauh
- Kontrol melalui browser
- Integrasi dengan server atau cloud
- Sistem IoT dan dashboard real-time

Pada bab ini, pembaca akan mempelajari dasar networking menggunakan Raspberry Pi serta membangun **web server sederhana** untuk mengontrol GPIO melalui browser.

9.1 Konsep Dasar Networking

Apa Itu Networking?

Networking adalah sistem yang memungkinkan dua atau lebih perangkat berkomunikasi melalui media tertentu (kabel atau wireless).

Dalam konteks Raspberry Pi, jaringan memungkinkan:

- Akses jarak jauh (SSH, VNC)
- Komunikasi client-server
- Kontrol berbasis web
- Pengiriman data sensor

Konsep Client-Server

Model paling umum dalam networking adalah **client-server**.

- **Server** → Perangkat yang menyediakan layanan
- **Client** → Perangkat yang meminta layanan

Dalam bab ini:

- Raspberry Pi bertindak sebagai **server**
- Laptop atau smartphone bertindak sebagai **client**
- Browser digunakan untuk mengirim request

Alamat IP

Setiap perangkat dalam jaringan memiliki **IP Address**.

Contoh:

192.168.1.101

IP ini digunakan untuk mengakses Raspberry Pi melalui browser:

Untuk mengetahui IP Raspberry Pi:

```
hostname -I
```

9.2 Mengakses Raspberry Pi via WiFi

Koneksi WiFi

Pastikan Raspberry Pi terhubung ke jaringan yang sama dengan laptop/smartphone.

Cek koneksi:

```
ping google.com
```

Jika ada respon, berarti koneksi internet aktif.

Akses SSH (Terminal Jarak Jauh)

Dari terminal komputer lain:

```
ssh pi@192.168.1.101
```

SSH memungkinkan pengendalian penuh tanpa monitor.

Akses Desktop (VNC)

Raspberry Pi juga dapat diakses secara grafis menggunakan VNC Viewer.

Ini umum digunakan dalam pengembangan sistem robot yang dipasang langsung di chassis.

9.3 Web Server Sederhana dengan Flask

Untuk membangun server web sederhana, kita menggunakan **Flask**, framework Python ringan.

Instalasi Flask

```
pip3 install flask
```

Konsep HTTP

Saat browser membuka halaman:

1. Browser mengirim **request**
2. Server memproses request
3. Server mengirim **response**

9.4 Eksperimen 1: Web Server Dasar

Tujuan

- Membuat server web di Raspberry Pi
- Mengaksesnya dari browser

Kode Program

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Web Server Raspberry Pi Aktif!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Dokumentasi Program

- Flask Instance

`app = Flask(__name__)` membuat objek server.

- Routing

`@app.route("/")` menentukan URL utama.

- Host 0.0.0.0

Artinya server dapat diakses dari perangkat lain dalam jaringan.

Cara Menjalankan

```
python3 app.py
```

Buka browser:

http://IP_RASPBERRY_PI:5000

Indikator Keberhasilan

- Browser menampilkan pesan
- Server dapat diakses dari smartphone
- Tidak ada error port binding

9.5 Kontrol GPIO via Browser

Sekarang kita integrasikan networking dengan GPIO.

Tujuan

- Mengontrol LED melalui browser
- Menghubungkan web interface dengan perangkat fisik

Asumsi Hardware

LED → GPIO 17

Kode Program

```
from flask import Flask
import RPi.GPIO as GPIO

app = Flask(__name__)

GPIO.setmode(GPIO.BCM)
```

```

LED = 17
GPIO.setup(LED, GPIO.OUT)

@app.route("/")
def home():
    return """
    <h1>Kontrol LED</h1>
    <a href="/on">LED ON</a><br>
    <a href="/off">LED OFF</a>
    """

@app.route("/on")
def led_on():
    GPIO.output(LED, GPIO.HIGH)
    return "LED MENYALA <br><a href='/'>Kembali</a>"

@app.route("/off")
def led_off():
    GPIO.output(LED, GPIO.LOW)
    return "LED MATI <br><a href='/'>Kembali</a>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

Dokumentasi Sistem

1. Arsitektur

Browser → HTTP Request → Flask → GPIO → LED

2. Routing

- / → Halaman utama
- /on → LED menyala

- /off → LED mati

3. Integrasi Hardware & Networking

Saat user klik link:

- Browser mengirim request
- Flask memanggil fungsi tertentu
- GPIO berubah status
- LED menyala/mati

Indikator Keberhasilan

- LED menyala saat klik ON
- LED mati saat klik OFF
- Bisa diakses via smartphone

9.6 Pengembangan: Dashboard Sederhana (Optional)

Tambahkan tombol HTML lebih interaktif:

```
return """
<h1>Dashboard EduPi Robo</h1>
<button onclick="location.href='/on'">ON</button>
<button onclick="location.href='/off'">OFF</button>
"""
```

9.7 Keamanan Dasar (Pengantar)

Dalam implementasi nyata:

- Jangan expose port tanpa firewall
- Gunakan autentikasi
- Gunakan HTTPS untuk produksi

Pada tahap ini, sistem masih berjalan dalam jaringan lokal (LAN).

Refleksi Pembelajaran

Pada bab ini, pembaca telah memahami:

- Konsep client–server
- IP Address dan akses jaringan
- Instalasi dan penggunaan Flask
- Integrasi Web dan GPIO
- Kontrol perangkat fisik melalui browser

Sekarang robot tidak lagi hanya bergerak secara lokal, tetapi sudah dapat dikontrol melalui jaringan.

Penutup Bab

Networking membuka pintu menuju IoT dan sistem terdistribusi.

BAB 10. IoT Fundamentals

Pada bab sebelumnya, pembaca telah membangun web server sederhana dan mengontrol GPIO melalui browser. Sistem tersebut sudah menunjukkan konsep **client-server** dalam jaringan lokal.

Namun dalam dunia nyata, perangkat tidak hanya dikontrol dari satu browser dalam satu jaringan. Perangkat IoT dapat:

- Mengirim data ke cloud
- Berkomunikasi dengan banyak client sekaligus
- Dipantau dari mana saja
- Terintegrasi dengan dashboard, database, atau sistem AI

Bab ini membahas fondasi **Internet of Things (IoT)** serta arsitektur komunikasi yang umum digunakan dalam sistem modern.

10.1 Apa Itu Internet of Things (IoT)?

Definisi IoT

Internet of Things (IoT) adalah konsep di mana perangkat fisik terhubung ke internet dan dapat:

- Mengirim data
- Menerima perintah
- Berinteraksi dengan sistem lain
- Beroperasi secara otomatis

Dalam konteks EduPi Robo:

- Raspberry Pi → IoT Device
- Sensor → Data Source
- Cloud / Server → Data Processing
- Dashboard → Monitoring Interface

10.2 Arsitektur Dasar IoT

IoT biasanya terdiri dari 4 lapisan utama:

1. Perception Layer

Sensor dan aktuator (ultrasonik, motor, kamera)

2. Network Layer

WiFi, Ethernet, MQTT, HTTP

3. Processing Layer

Server, Cloud, Database

4. Application Layer

Dashboard, Monitoring App, Mobile Interface

Pendekatan ini membantu memahami bahwa IoT bukan hanya perangkat, tetapi sistem terintegrasi.

10.3 HTTP vs MQTT

Dalam IoT, dua protokol populer adalah **HTTP** dan **MQTT**.

1. HTTP (Request–Response)

Digunakan saat:

- Browser meminta data
- REST API digunakan
- Sistem berbasis web

Karakteristik:

- Client harus meminta data
- Tidak real-time push
- Lebih berat (overhead besar)

2. MQTT (Publish–Subscribe)

MQTT adalah protokol ringan untuk IoT.

Karakteristik:

- Menggunakan broker
- Model publish–subscribe
- Lebih ringan dari HTTP
- Cocok untuk perangkat embedded

10.4 Konsep Publish-Subscribe Model

Dalam MQTT terdapat tiga komponen:

- Publisher → Mengirim data
- Subscriber → Menerima data
- Broker → Penghubung pesan

Contoh:

Sensor suhu → publish ke topic "sensor/suhu"

Dashboard → subscribe ke topic "sensor/suhu"

Broker akan mengirimkan pesan ke semua subscriber yang terdaftar.

10.5 Instalasi MQTT Broker (Mosquitto)

Instalasi di Raspberry Pi

```
sudo apt update  
sudo apt install mosquitto mosquitto-clients -y
```

Aktifkan service:

```
sudo systemctl enable mosquitto
```

Cek status:

```
sudo systemctl status mosquitto
```

10.6 Eksperimen 1: Publish-Subscribe Lokal

Terminal 1 (Subscriber)

```
mosquitto_sub -t sensor/test
```

Terminal 2 (Publisher)

```
mosquitto_pub -t sensor/test -m "Halo IoT"
```

Jika subscriber menerima pesan, berarti broker berjalan dengan baik.

10.7 Mengirim Data Sensor ke MQTT dengan Python

Instalasi Library

```
pip3 install paho-mqtt
```

Contoh Program Publish Data Sensor

```
import paho.mqtt.client as mqtt
import time
import random

broker = "localhost"
topic = "robot/sensor"

client = mqtt.Client()
client.connect(broker, 1883, 60)

try:
    while True:
        # Simulasi data jarak
        jarak = random.randint(10, 100)

        pesan = f"Jarak: {jarak} cm"
        client.publish(topic, pesan)
        print("Mengirim:", pesan)

        time.sleep(2)

except KeyboardInterrupt:
    print("Program dihentikan")

finally:
    client.disconnect()
```

Dokumentasi Program

1. MQTT Client

Membuat objek client dan menghubungkannya ke broker.

2. Topic

Topic digunakan sebagai “alamat pesan”.

3. Publish

Data dikirim ke broker setiap 2 detik.

4. Disconnect

Koneksi ditutup dengan aman saat program berhenti.

10.8 Program Subscriber dengan Python

```
import paho.mqtt.client as mqtt

broker = "localhost"
topic = "robot/sensor"
def on_message(client, userdata, message):
    print("Pesan diterima:",
          str(message.payload.decode()))

client = mqtt.Client()
client.on_message = on_message

client.connect(broker, 1883, 60)
client.subscribe(topic)
```

```
client.loop_forever()
```

Dokumentasi Subscriber

- `on_message()` → dipanggil saat pesan diterima
- `subscribe()` → mendaftar pada topic
- `loop_forever()` → menjaga koneksi tetap aktif

10.9 Integrasi Sensor Nyata ke MQTT

Contoh integrasi sensor ultrasonik:

```
jarak = baca_jarak()  
client.publish("robot/jarak", jarak)
```

Sekarang data sensor dapat:

- Dipantau di dashboard
- Disimpan di database
- Digunakan untuk AI decision system

10.10 Kelebihan MQTT untuk Robotika

- Ringan
- Real-time push
- Banyak client bisa menerima data
- Cocok untuk sistem terdistribusi

Contoh penggunaan:

- Monitoring robot via cloud
- Kontrol robot dari aplikasi mobile
- Logging data navigasi

Refleksi Pembelajaran

Pada bab ini, pembaca telah memahami:

- Konsep dasar IoT
- Arsitektur IoT modern
- Perbedaan HTTP dan MQTT
- Publish–subscribe model
- Instalasi dan penggunaan Mosquitto
- Integrasi sensor ke MQTT

Sekarang Raspberry Pi bukan hanya perangkat lokal, tetapi telah menjadi **node IoT** dalam jaringan.

Penutup Bab

IoT membuka kemungkinan tak terbatas:

- Robot yang dapat dipantau dari mana saja
- Sistem sensor yang terintegrasi cloud
- Dashboard real-time
- Integrasi AI berbasis data

BAB 11. Remote Control & Monitoring System pada Edupi

Pada bab sebelumnya, kita telah memahami konsep IoT, MQTT, dan arsitektur komunikasi modern. Pada bab ini, kita akan membahas bagaimana **Edupi** mengimplementasikan sistem kontrol dan monitoring real-time berbasis WebSocket dan Web Interface.

Bab ini menjadi jembatan antara:

- Networking
- Hardware Control
- AI Processing
- Real-time Dashboard

Edupi bukan sekadar robot yang terhubung ke jaringan, tetapi sebuah **sistem terintegrasi berbasis web dengan kontrol real-time**.

11.1 Arsitektur Remote Control Edupi

Arsitektur sistem Edupi memiliki tiga lapisan utama:

1. Client Layer

- Browser (Laptop / Smartphone)
- Web-based controller
- Dashboard monitoring

2. Application Layer

- FastAPI Server (Port 8000)
- WebSocket endpoints
- Video streaming MJPEG
- Mode AI switching

3. Hardware Layer

- Motor 4WD
- Servo pan/tilt
- Ultrasonik
- Line follower 5-channel
- Kamera + AI
- LED & buzzer

Komunikasi utama menggunakan:

- HTTP (untuk halaman web & video)
- WebSocket (untuk kontrol real-time)

11.2 Konsep WebSocket pada Edupi

Berbeda dengan HTTP yang bersifat request–response, WebSocket bersifat:

- Full-duplex (dua arah)
- Real-time
- Persisten connection

Artinya:

Browser tidak perlu terus-menerus melakukan refresh.
Server dapat langsung mengirim data sensor ke client.

Contoh Endpoint Edupi:

/ws/control
/ws/autoPilot
/ws/lineFollower
/ws/objectDetect
/ws/gestureControl

11.3 Studi Kasus 1: Manual Control Mode

Tujuan

Mengontrol robot Edupi secara manual melalui joystick web.

Contoh Payload WebSocket

```
{  
  "cmd": "move",  
  "x": 0.0,  
  "y": 0.5,  
  "speed": 80  
}
```

Dokumentasi Parameter

Parameter	Fungsi
x	Steering (-1.0 s/d 1.0)

y	Throttle (-1.0 s/d 1.0)
speed	Limit kecepatan (%)

Mekanisme Internal Edupi

1. Web client mengirim JSON via WebSocket
2. FastAPI menerima data
3. Data diproses di motor module
4. PWM dikalkulasi
5. Motor 4WD bergerak

Konsep Differential Steering

Motor kiri dan kanan dikontrol terpisah.

Jika:

- $x = 0 \rightarrow$ lurus
- $x > 0 \rightarrow$ belok kanan
- $x < 0 \rightarrow$ belok kiri

Speed akhir dihitung sebagai kombinasi x dan y.

11.4 Studi Kasus 2: Monitoring Sensor Real-Time

Edupi mengirim data sensor melalui WebSocket broadcast.

Contoh data:

```
{  
  "distance": 25.5,  
  "line_sensors": [0,0,1,0,0],  
  "emergency": false,  
  "battery": 7.2  
}
```

Dokumentasi Field

Field	Deskripsi
distance	Jarak ultrasonik (cm)
line_sensors	Array 5 channel IR
emergency	Sensor darurat
battery	Tegangan baterai

Alur Monitoring

Sensor → Application Layer → Broadcast → Dashboard

Data dapat:

- Ditampilkan dalam grafik
- Disimpan ke database
- Digunakan untuk AI decision

11.5 Studi Kasus 3: Auto Pilot Mode

Mode ini menggunakan:

- Ultrasonik
- AI logic
- Motor control

Contoh command:

```
{  
  "cmd": "toggle",  
  "state": true,  
  "speed": 50,  
  "follow_distance": 30  
}
```

Logika Internal

Jika jarak < follow_distance:

- Robot berhenti / belok

Jika jarak > follow_distance:

- Robot maju

Ini adalah pengembangan dari Bab Navigasi sebelumnya, tetapi kini berjalan dalam sistem real-time berbasis WebSocket.

11.6 Studi Kasus 4: Line Follower dengan PID

Edupi memiliki 5 channel IR sensor.

Array contoh:

[0, 0, 1, 0, 0]

Artinya garis berada di tengah.

PID digunakan untuk mengoreksi error posisi.

Contoh payload:

```
{  
  "cmd": "toggle",  
  "state": true,  
  "speed": 40,  
  "kp": 0.8,  
  "ki": 0.0,  
  "kd": 0.2  
}
```

Konsep PID

Error = posisi garis – posisi tengah

Output = $(kp \times \text{error}) + (kd \times \text{delta error})$

Motor kiri dan kanan dikoreksi berdasarkan output PID.

11.7 Studi Kasus 5: AI Mode (Object Detection)

Edupi menggunakan TensorFlow Lite untuk mendeteksi 11 objek

Contoh payload:

```
{  
  "cmd": "toggle",  
  "state": true,  
  "target": "person"  
}
```

Jika target terdeteksi:

- Servo pan/tilt mengikuti objek
- Robot dapat berhenti atau mendekat

11.8 Integrasi Multi-Mode System

Edupi memiliki beberapa mode:

- Manual
- Auto Pilot
- Line Follower
- Color Tracking
- Face Tracking
- Object Detection
- Gesture Control

Sistem memastikan:

Hanya satu mode aktif dalam satu waktu.

Hal ini mencegah konflik kontrol motor.

11.9 Keamanan dan Manajemen Sistem

EduPi memiliki:

- Manager Service (Port 5000)
- Start / Stop robot
- Reboot system
- Crash logging

Ini penting dalam sistem robot yang berjalan tanpa monitor.

Refleksi Pembelajaran

Pada bab ini, pembaca telah memahami:

- Arsitektur remote control EduPi
- WebSocket real-time communication
- Monitoring sensor dashboard
- Integrasi AI dan motor control
- Konsep multi-mode robot system

EduPi kini bukan hanya robot bergerak, tetapi sebuah:

Platform robotika cerdas berbasis web dan AI.

Penutup Bab

Dengan menguasai sistem remote control dan monitoring, pembaca telah memahami bagaimana:

- Hardware
- Networking
- IoT
- AI
- Web Technology

terintegrasi dalam satu sistem nyata.

BAB 12. Mini Project Intermediate

Pada bab ini, seluruh konsep yang telah dipelajari pada level Intermediate akan digabungkan menjadi sistem yang utuh pada platform **Edupi**.

Konsep yang digunakan meliputi:

- Kendali motor 4WD
- Sensor ultrasonik
- Line sensor (opsional)
- Web-based control (WebSocket / HTTP)
- MQTT untuk monitoring
- Arsitektur modular

Bab ini terdiri dari tiga studi kasus utama:

1. Robot Penghindar Halangan (Autonomous Mode)
2. Robot Remote Control via Web
3. IoT Monitoring Robot

Setiap proyek akan disertai tujuan, arsitektur sistem, implementasi, serta dokumentasi teknis.

12.1 Proyek 1 — Robot Penghindar Halangan Tujuan

Membangun robot Edupi yang mampu:

- Bergerak maju secara otomatis
- Mendeteksi objek di depan
- Menghindari tabrakan
- Mengambil keputusan berbasis jarak

Arsitektur Sistem

Sensor Ultrasonik → Decision Logic → Motor 4WD

Sistem bekerja secara autonomous tanpa intervensi user.

Spesifikasi Sistem

- Sensor HC-SR04 di depan robot
- Threshold jarak: 30 cm
- Jika jarak < 30 cm → belok
- Jika jarak $\geq$ 30 cm → maju

Implementasi Program (Standalone Trainer Mode)

```
import time
from modules.motor import MotorDriver
from modules.sensors import UltrasonicSensor

motor = MotorDriver()
ultra = UltrasonicSensor(trigger_pin=24, echo_pin=20)

SAFE_DISTANCE = 30
try:
    while True:
        jarak = ultra.read_distance()
        print("Jarak:", jarak)

        if jarak > SAFE_DISTANCE:
            motor.forward(speed=50)
        else:
            motor.stop()
            time.sleep(0.3)
            motor.turn_right(speed=50)
            time.sleep(0.6)

except KeyboardInterrupt:
    print("Program dihentikan")

finally:
    motor.stop()
```

Dokumentasi Program

1. MotorDriver()

Mengontrol 4 motor DC secara terintegrasi.

2. UltrasonicSensor()

Mengembalikan nilai jarak dalam centimeter.

3. SAFE_DISTANCE

Menentukan batas aman navigasi.

4. Decision Logic

If-Else berbasis rule sederhana (open-loop).

Evaluasi Sistem

Perhatikan:

- Apakah robot bergerak lurus?
- Apakah belokan cukup untuk menghindari?
- Apakah ada delay pembacaan sensor?

Optimasi dapat dilakukan dengan:

- Filter pembacaan jarak (moving average)
- Variasi arah belok (kiri/kanan bergantian)

12.2 Proyek 2 — Robot Remote Control via Web

Tujuan

Mengendalikan Edupi melalui:

- Browser laptop
- Smartphone
- WebSocket real-time

Arsitektur Sistem

Browser → WebSocket → FastAPI → MotorDriver

Konsep Kendali

Steering & Throttle:

- y → maju/mundur
- x → belok

Contoh Handler WebSocket (Sederhana)

```
from fastapi import WebSocket
from modules.motor import MotorDriver

motor = MotorDriver()

async def websocket_control(websocket: WebSocket):
    await websocket.accept()
    try:
```

```

while True:
    data = await websocket.receive_json()

    if data["cmd"] == "move":
        x = data["x"]
        y = data["y"]
        speed = data["speed"]

        motor.drive(x, y, speed)

    elif data["cmd"] == "stop":
        motor.stop()

except:
    motor.stop()

```

Dokumentasi Sistem

“drive(x, y, speed)”

Fungsi ini:

- Menghitung kecepatan motor kiri & kanan
- Menggunakan differential steering
- Mengatur PWM sesuai parameter

Evaluasi

Uji:

- Respons latency
- Kestabilan koneksi WiFi
- Pergerakan halus atau patah-patah

Optimasi:

- Tambahkan speed smoothing
- Tambahkan deadzone joystick

12.3 Proyek 3 — IoT Monitoring Robot

Tujuan

Mengirim data sensor Edupi ke broker MQTT untuk:

- Monitoring jarak
- Monitoring line sensor
- Monitoring status robot

Arsitektur IoT

Sensor → MQTT Publish → Broker → Dashboard / Subscriber

Implementasi Publisher

```
import paho.mqtt.client as mqtt
import time
from modules.sensors import UltrasonicSensor

client = mqtt.Client()
client.connect("localhost", 1883, 60)

ultra = UltrasonicSensor(24, 20)

try:
    while True:
        jarak = ultra.read_distance()
```

```
        client.publish("edupi/distance", jarak)
        print("Publish:", jarak)
        time.sleep(2)

except KeyboardInterrupt:
    pass

finally:
    client.disconnect()
```

Dokumentasi Program

1. Topic

edupi/distance → digunakan untuk monitoring jarak

2. Interval

Data dikirim setiap 2 detik

3. Broker

Dapat menggunakan:

- Mosquitto lokal
- Cloud MQTT (HiveMQ, EMQX, dll)

Pengembangan Dashboard

Data dapat:

- Ditampilkan dalam grafik real-time
- Disimpan dalam database
- Digunakan untuk logging perjalanan robot

12.4 Integrasi Tiga Sistem

Pada tahap akhir, ketiga sistem dapat digabungkan:

- Auto Mode → obstacle avoidance
- Manual Mode → web control
- Monitoring → MQTT telemetry

Pastikan hanya satu mode motor aktif pada satu waktu untuk mencegah konflik kontrol.

Struktur Sistem Final Edupi (Intermediate)

1. Hardware Layer
Motor, sensor, kamera
2. Control Layer
MotorDriver, SensorModule
3. Application Layer
FastAPI + WebSocket
4. IoT Layer
MQTT publish/subscrib

Refleksi Pembelajaran Level Intermediate

Setelah menyelesaikan Bab 7–12, pembaca telah mampu:

- Mengendalikan motor 4WD
- Membaca sensor navigasi
- Membangun obstacle avoidance
- Mengembangkan kontrol berbasis web
- Mengirim data ke sistem IoT
- Mengintegrasikan hardware & networking

Ini adalah fondasi kuat sebelum memasuki Level Advanced (AI & Computer Vision).

Penutup Bab

Mini Project Intermediate membuktikan bahwa Edupi bukan sekadar trainer kit, tetapi sebuah:

Platform robotika modular

Siap dikembangkan

Siap diintegrasikan dengan AI

Pada level berikutnya, kita akan memasuki:

LEVEL 3 — ADVANCED

AI, Computer Vision, dan Intelligent System.

LEVEL 3 — ADVANCED

AI, Computer Vision, dan Intelligent System.

Pada level sebelumnya, Edupi telah mampu:

- Bergerak secara autonomous
- Dikendalikan melalui web
- Mengirim data melalui IoT
- Mengintegrasikan sensor dan aktuator

Pada Level Advanced ini, kita memasuki tahap baru:

Edupi tidak hanya merespons sensor — tetapi mampu “melihat” dan mengambil keputusan berbasis AI.

Fokus utama level ini adalah:

- AI on the Edge
- Computer Vision
- Decision Making System
- Optimasi performa pada Raspberry Pi
-

Level ini sangat relevan untuk:

- Riset robotika
- Kompetisi robot
- Pengembangan produk
- Prototipe sistem cerdas

BAB 14. Pengenalan AI di Raspberry Pi

Bab ini membahas fondasi konseptual sebelum masuk ke implementasi Computer Vision pada Edupi.

14.1 AI vs ML vs Deep Learning

Dalam banyak diskusi, istilah AI, Machine Learning, dan Deep Learning sering digunakan secara bergantian. Padahal ketiganya memiliki perbedaan mendasar.

1. Artificial Intelligence (AI)

AI adalah konsep besar.

AI adalah kemampuan sistem untuk:

- Mengambil keputusan
- Menyelesaikan masalah
- Meniru kecerdasan manusia

Contoh pada Edupi:

- Robot berhenti saat mendeteksi orang
- Robot mengikuti wajah
- Robot mengenali objek tertentu

AI dapat berupa:

- Rule-based system

- Machine Learning
- Deep Learning

2. Machine Learning (ML)

Machine Learning adalah bagian dari AI.

ML memungkinkan sistem:

- Belajar dari data
- Menghasilkan model prediktif
- Mengambil keputusan berbasis pola

Contoh:

- Model klasifikasi warna
- Model prediksi arah garis
- Model pengenalan gesture

ML tidak selalu menggunakan jaringan saraf.

3. Deep Learning (DL)

Deep Learning adalah bagian dari Machine Learning.

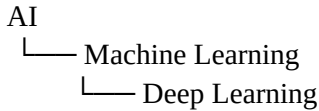
DL menggunakan:

- Neural Network berlapis (deep neural network)
- CNN (Convolutional Neural Network)
- Model vision seperti SSD, YOLO, MobileNet

Contoh pada Edupi:

- Object Detection dengan TFLite
- Face Detection dengan MediaPipe
- Gesture Recognition

Ringkasan Hierarki



14.2 Edge AI Concept

Apa Itu Edge AI?

Edge AI adalah konsep menjalankan model AI langsung pada perangkat (edge device), tanpa bergantung pada cloud.

Dalam konteks Edupi:

- Model berjalan langsung di Raspberry Pi
- Tidak perlu internet
- Latensi rendah
- Privasi data lebih aman

Perbandingan Cloud AI vs Edge AI

Aspek	Cloud AI	Edge AI
Latensi	Tinggi	Rendah
Internet	Wajib	Tidak wajib
Privasi	Data dikirim	Data lokal

Skalabilitas	Tinggi	Terbatas
--------------	--------	----------

Edupi menggunakan pendekatan **Edge AI**, karena:

- Robot harus merespons cepat
- Tidak selalu terhubung internet
- Digunakan untuk pembelajaran offline

14.3 Keterbatasan Hardware Raspberry Pi

Raspberry Pi bukan GPU server.

Beberapa keterbatasan penting:

1. CPU Terbatas

- ARM-based processor
- Tidak setara dengan desktop GPU

2. RAM Terbatas

- 2GB / 4GB / 8GB
- Model besar dapat menyebabkan swap

3. Tidak Ada GPU AI Dedicated

- Tidak ada CUDA
- Inferensi murni CPU (kecuali menggunakan Coral / NPU tambahan)

Dampak pada Sistem Edupi

- Model harus ringan
- Resolusi kamera tidak terlalu besar
- Frame rate perlu dibatasi
- Harus ada optimasi

14.4 Optimasi Model untuk Edge Device

Agar AI berjalan lancar pada Edupi, beberapa teknik optimasi digunakan.

1. Gunakan TensorFlow Lite (TFLite)

TFLite adalah versi ringan TensorFlow untuk embedded system.

Keunggulan:

- Ukuran model lebih kecil
- Inferensi lebih cepat
- Dioptimalkan untuk ARM

2. Gunakan Model Ringan

Contoh model ringan:

- MobileNet SSD
- EfficientNet Lite
- MediaPipe

Hindari model berat seperti:

- YOLOv5 full
- ResNet besar

3. Quantization

Quantization mengubah model dari:

- Float32 → Int8

Keuntungan:

- Ukuran model lebih kecil
- Inferensi lebih cepat
- Sedikit penurunan akurasi

4. Batasi Resolusi Frame

Contoh:

640x480 → 320x240

Keuntungan:

- Lebih sedikit pixel diproses
- FPS meningkat

5. Frame Skipping

Tidak semua frame perlu diproses AI.

Contoh:

- Proses 1 dari setiap 3 frame
- Frame lain hanya ditampilkan

Ini meningkatkan performa signifikan.

6. Gunakan Threading / Async

Pisahkan:

- Thread kamera
- Thread AI inference
- Thread motor control

Agar sistem tidak blocking.

14.5 Studi Kasus: Struktur AI Pipeline pada Edupi

Pipeline sederhana:

1. Capture frame dari kamera
2. Resize frame
3. Pre-processing (normalisasi)
4. Inferensi model
5. Post-processing (bounding box, label)
6. Decision logic (gerak robot)

Contoh pseudo-code pipeline:

```
while True:
    frame = camera.read()
    resized = preprocess(frame)
    result = model.infer(resized)
    objects = postprocess(result)

    if "person" in objects:
        motor.stop()
```

14.6 Tantangan Implementasi AI pada Robot

Beberapa tantangan nyata:

- Overheating CPU
- Frame drop
- Latensi tinggi
- Konflik akses kamera
- Konsumsi daya meningkat

Solusi umum:

- Gunakan heatsink + fan
- Batasi FPS
- Optimasi model
- Gunakan mode AI terpisah

Refleksi Pembelajaran

Pada bab ini, pembaca telah memahami:

- Perbedaan AI, ML, dan Deep Learning
- Konsep Edge AI
- Keterbatasan Raspberry Pi
- Teknik optimasi model
- Struktur pipeline AI pada Edupi

Pemahaman ini penting sebelum masuk ke implementasi Computer Vision pada bab berikutnya.

Penutup Bab

Edupi kini siap memasuki dunia Computer Vision.

BAB 15. Computer Vision Dasar pada Edupi

Pada bab sebelumnya kita telah memahami konsep AI on the Edge serta keterbatasan hardware Raspberry Pi. Sekarang kita mulai masuk ke implementasi Computer Vision secara langsung pada Edupi.

Bab ini akan membahas:

- Kamera Raspberry Pi
- Pengenalan OpenCV
- Image capture & video streaming
- Pre-processing image

Di setiap bagian akan dijelaskan metode akses yang disarankan:

- **SSH saja** → jika hanya menjalankan program tanpa tampilan
- **VNC / Desktop** → jika membutuhkan preview GUI (cv2.imshow)
- **Web Browser** → jika menggunakan streaming MJPEG dari server Edupi

15.1 Kamera Raspberry Pi pada Edupi

Edupi dapat menggunakan:

- Pi Camera (CSI Camera)
- USB Camera

Kamera menjadi sensor utama untuk AI Vision Mode seperti:

- Object Detection
- Face Detection
- Gesture Recognition
- Color Tracking

Cara Akses Kamera

1. Cek Kamera (SSH saja)

Gunakan terminal:

```
vcgencmd get_camera
```

Jika menggunakan USB:

```
ls /dev/video*
```

2. Test Capture Cepat (SSH saja)

```
libcamera-still -o test.jpg
```

Jika file test.jpg berhasil dibuat → kamera aktif.

15.2 OpenCV Introduction

OpenCV adalah library utama untuk pengolahan citra pada Edupi.

Digunakan untuk:

- Capture frame
- Resize image
- Konversi warna
- Filtering
- Deteksi objek

Instalasi OpenCV (SSH saja)

```
sudo apt install python3-opencv
```

atau

```
pip install opencv-python
```

15.3 Image Capture & Video Stream

1. Capture Frame Sederhana

*Akses yang disarankan: VNC atau Desktop, Karena kita akan menggunakan cv2.imshow().

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    cv2.imshow("Edupi Camera", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Dokumentasi Program

- VideoCapture(0) → membuka kamera default
- read() → mengambil frame
- imshow() → menampilkan frame
- waitKey() → kontrol keyboard

Catatan Penting

Jika dijalankan melalui SSH tanpa GUI, akan muncul error karena tidak ada display server.

Solusi:

- Gunakan VNC
- Atau jalankan tanpa imshow

2. Capture Tanpa GUI (SSH saja)

Jika hanya ingin memproses frame tanpa menampilkan:

```
import cv2

cap = cv2.VideoCapture(0)
ret, frame = cap.read()

if ret:
    cv2.imwrite("capture.jpg", frame)

cap.release()
```

Cocok untuk:

- AI processing backend
- Integrasi dengan server

3. Video Stream via Web (Web Browser)

Jika Edupi menggunakan sudah FastAPI + MJPEG streaming, akses melalui:

http://IP_EDUPI:8000/video_feed

15.4 Pre-processing Image

Sebelum masuk ke model AI, gambar harus diproses terlebih dahulu.

Tujuan pre-processing:

- Mengurangi noise
- Mengubah ukuran
- Mengubah format warna
- Normalisasi pixel

Contoh 1: Resize Frame

```
resized = cv2.resize(frame, (320, 240))
```

Manfaat:

- Mengurangi beban CPU
- Meningkatkan FPS

Contoh 2: Convert ke Grayscale

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Digunakan untuk:

- Haar Cascade
- Edge detection

Contoh 3: Gaussian Blur

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

Manfaat:

- Mengurangi noise
- Meningkatkan stabilitas deteksi

Contoh Pipeline Sederhana

```
frame = cap.read()[1]  
resized = cv2.resize(frame, (320,240))  
gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)  
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

Refleksi Bab 15

Pada bab ini, pembaca telah memahami:

- Cara mengakses kamera
- Perbedaan akses SSH / VNC / Web
- Dasar penggunaan OpenCV
- Pipeline pre-processing

Edupi kini sudah mampu menangkap dan memproses citra.

BAB 16. Face Detection & Recognition

Bab ini membawa Edupi ke tahap baru:

Robot mampu mendeteksi dan mengenali wajah manusia.

16.1 Haar Cascade vs DNN

Haar Cascade

Metode klasik berbasis:

- Haar-like features
- Cascade classifier

Keunggulan:

- Ringan
- Cepat di CPU
- Cocok untuk Raspberry Pi

Kekurangan:

- Kurang akurat
- Sensitif terhadap pencahayaan

DNN (Deep Neural Network)

Menggunakan model CNN modern.

Keunggulan:

- Lebih akurat
- Robust terhadap variasi wajah

Kekurangan:

- Lebih berat
- Butuh optimasi

16.2 Face Detection Pipeline

Pipeline umum:

1. Capture frame
2. Resize
3. Convert ke grayscale
4. Jalankan detector
5. Gambar bounding box
6. Decision logic

Implementasi Haar Cascade

Akses: VNC (karena menggunakan imshow)

```
import cv2
```

```

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)
cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(
        gray,
        scaleFactor=1.3,
        minNeighbors=5
    )

    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x,y), (x+w,y+h),
            (0,255,0), 2)

    cv2.imshow("Face Detection - Edupi", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()

cv2.destroyAllWindows()

```

Dokumentasi Program

- detectMultiScale() → mendeteksi wajah
- scaleFactor → sensitivitas deteksi
- minNeighbors → stabilitas bounding box

16.3 Face Recognition Konsep

Face Detection $\neq$ Face Recognition

- Detection $\rightarrow$ mendeteksi ada wajah
- Recognition $\rightarrow$ mengenali siapa orangnya

Konsep Face Recognition

Langkah umum:

1. Deteksi wajah
2. Ekstraksi fitur wajah (embedding)
3. Bandingkan dengan database
4. Tentukan identitas

Metode umum:

- LBPH
- FaceNet
- MediaPipe + classifier

16.4 Aplikasi: Deteksi Wajah Menggunakan Kamera

Pada bagian ini, kita akan membangun sistem sederhana yang mampu:

- Mengambil video dari kamera

- Mendeteksi wajah secara real-time
- Menampilkan bounding box pada wajah yang terdeteksi

Tujuan utama bagian ini adalah memahami alur dasar:

Capture → Pre-processing → Detection → Visualisasi

Tanpa integrasi aktuator atau sistem robot.

Tujuan Praktikum

Setelah menyelesaikan bagian ini, peserta mampu:

- Mengakses kamera menggunakan OpenCV
- Menggunakan Haar Cascade untuk mendeteksi wajah
- Memahami parameter deteksi
- Menampilkan hasil deteksi dalam bentuk bounding box

Akses yang Disarankan

Karena kita menggunakan `cv2.imshow()` untuk menampilkan video:

Gunakan VNC / Desktop GUI

Jika dijalankan melalui SSH tanpa GUI, akan muncul error display.

Alternatif (tanpa GUI) akan dijelaskan di bagian akhir.

Konsep Sistem

Alur kerja program:

1. Buka kamera
2. Ambil frame
3. Ubah ke grayscale
4. Jalankan deteksi wajah
5. Gambar kotak pada wajah
6. Tampilkan hasil

Program Face Detection Dasar (Haar Cascade)

```
import cv2

# Load model Haar Cascade bawaan OpenCV
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

# Buka kamera
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Convert ke grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Deteksi wajah
```

```

faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.3,
    minNeighbors=5
)

# Gambar bounding box
for (x, y, w, h) in faces:
    cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)

# Tampilkan hasil
cv2.imshow("Face Detection - Trainer Kit", frame)

# Tekan q untuk keluar
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Dokumentasi Program

1. Load Haar Cascade

```
cv2.CascadeClassifier(...)
```

Memuat model deteksi wajah yang telah dilatih sebelumnya.

2. VideoCapture(0)

Membuka kamera default.

Jika menggunakan USB kedua:

```
cv2.VideoCapture(1)
```

3. Convert ke Grayscale

```
cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Haar Cascade bekerja lebih cepat pada citra grayscale.

4. detectMultiScale()

Parameter penting:

- `scaleFactor`
Mengontrol skala pencarian wajah.
Nilai kecil → lebih sensitif (lebih lambat).

- `minNeighbors`
Mengontrol stabilitas deteksi.
Nilai tinggi → lebih sedikit false positive.

5. Bounding Box

```
cv2.rectangle(...)
```

Menggambar kotak di sekitar wajah yang terdeteksi.

Eksperimen Tambahan

Coba ubah parameter berikut dan amati hasilnya:

```
scaleFactor=1.1  
minNeighbors=3
```

Perhatikan:

- Apakah deteksi lebih sensitif?
- Apakah muncul false detection?

Jika Tidak Menggunakan VNC (SSH Saja)

Jika hanya ingin mengetahui apakah wajah terdeteksi tanpa GUI:

```
import cv2  
  
face_cascade = cv2.CascadeClassifier(  
    cv2.data.haarcascades +  
    "haarcascade_frontalface_default.xml"  
)  
  
cap = cv2.VideoCapture(0)
```

```
ret, frame = cap.read()

if ret:
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.3,
5)

    print("Jumlah wajah terdeteksi:", len(faces))

cap.release()
```

Cocok untuk:

- Backend processing
- Logging
- Integrasi server

Tidak perlu VNC.

Evaluasi Hasil

Sistem dianggap berhasil jika:

- Wajah manusia terdeteksi dengan bounding box
- Deteksi stabil saat kepala bergerak
- Tidak banyak false detection pada objek lain

Tantangan Umum

1. Pencahayaan terlalu gelap → deteksi gagal
2. Wajah terlalu jauh → tidak terdeteksi
3. CPU tinggi jika resolusi terlalu besar

Solusi:

- Gunakan resize frame (320x240)
- Tambahkan pencahayaan
- Kurangi scaleFactor

Refleksi Pembelajaran

Melalui praktikum ini, peserta telah memahami:

- Dasar Computer Vision real-time
- Cara kerja model Haar Cascade
- Konsep pipeline deteksi wajah
- Pengaruh parameter terhadap performa

Ini adalah fondasi penting sebelum melanjutkan ke:

- Face Tracking
- Face Recognition
- Object Detection
- AI berbasis DNN

Penutup

Pada tahap ini, Trainer Kit telah mampu:

Menangkap video
Mendeteksi wajah
Menampilkan hasil secara real-time

Langkah berikutnya pada bab selanjutnya adalah memahami bagaimana:

- Meningkatkan akurasi dengan DNN
- Mengenali identitas wajah
- Mengembangkan sistem vision yang lebih cerdas

BAB 17. Object Detection

Pada bab sebelumnya kita telah belajar mendeteksi wajah menggunakan Haar Cascade. Sekarang kita naik satu tingkat: bukan hanya wajah, tetapi berbagai objek seperti orang, botol, kursi, dan sebagainya. Inilah yang disebut **Object Detection**.

17.1 Konsep Object Detection

Object Detection adalah proses:

1. Mendeteksi objek dalam gambar
2. Menentukan lokasi objek (bounding box)
3. Mengklasifikasikan objek tersebut

Berbeda dengan image classification yang hanya memberi satu label untuk seluruh gambar, object detection mampu:

- Menemukan banyak objek sekaligus
- Memberi label berbeda untuk tiap objek

- Menunjukkan posisi objek

Contoh hasil deteksi:

- person (0.92)
- bottle (0.87)
- chair (0.78)

17.2 YOLO / SSD / MobileNet (Overview)

Beberapa model populer dalam object detection:

1. YOLO (You Only Look Once)

- Sangat cepat
- Akurasi tinggi
- Cocok untuk GPU
- Versi penuh terlalu berat untuk Raspberry Pi

2. SSD (Single Shot Detector)

- Lebih ringan dari YOLO
- Cocok untuk embedded system
- Umumnya dipadukan dengan MobileNet

3. MobileNet

- Arsitektur CNN ringan
- Dioptimalkan untuk perangkat kecil
- Sering digunakan bersama SSD

Untuk Raspberry Pi, kombinasi yang umum digunakan adalah:

SSD MobileNet (TensorFlow Lite)

Karena ringan dan cukup akurat.

17.3 Implementasi Model Ringan (TensorFlow Lite)

Akses yang disarankan:

- Jalankan via SSH
- Jika ingin melihat bounding box → gunakan VNC atau streaming web

Struktur Dasar Pipeline

1. Capture frame
2. Resize ke ukuran model
3. Jalankan inferensi
4. Ambil hasil deteksi
5. Gambar bounding box

Contoh Pseudocode Sederhana

```
frame = camera.read()
input_data = preprocess(frame)
result = model.invoke(input_data)
for obj in result:
    if obj["score"] > 0.5:
        draw_box(frame, obj)
```

Prinsip Penting

- Gunakan resolusi kecil (misal 320x320)
- Batasi FPS
- Gunakan model TFLite yang sudah quantized (INT8)

17.4 Real-time Object Detection

Jika ingin menampilkan hasil secara langsung:

Gunakan VNC (untuk imshow)

atau

Gunakan web streaming (jika server sudah ada)

Contoh sederhana dengan bounding box:

```
for (x, y, w, h) in detected_objects:
    cv2.rectangle(frame, (x,y), (x+w,y+h), (255,0,0),
2)
```

Evaluasi Praktek

Perhatikan:

- FPS stabil atau tidak
- CPU usage tinggi atau tidak
- Apakah deteksi konsisten

Refleksi Bab 17

Peserta telah memahami:

- Konsep object detection
- Perbedaan YOLO, SSD, MobileNet
- Mengapa model ringan diperlukan
- Pipeline dasar object detection

BAB 18. Gesture Recognition

Setelah objek dan wajah, sekarang kita masuk ke gesture (gerakan tangan).

Gesture Recognition memungkinkan sistem:

- Mengenali posisi tangan
- Mengidentifikasi gesture tertentu
- Menggunakan gesture sebagai input kontrol

18.1 Konsep Gesture Recognition

Gesture recognition bekerja dengan:

1. Mendeteksi tangan
2. Melacak titik-titik (landmark) jari
3. Menghitung pola posisi jari
4. Menentukan jenis gesture

Contoh gesture:

- Open palm
- Fist
- Thumb up
- Two fingers

18.2 Hand Tracking Dasar

Model populer untuk hand tracking:

- MediaPipe Hands

Model ini ringan dan cocok untuk Raspberry Pi.

Pipeline Dasar

1. Capture frame
2. Convert ke RGB
3. Jalankan model MediaPipe
4. Ambil landmark tangan
5. Visualisasi titik tangan

Contoh Konsep Landmark

MediaPipe mendeteksi 21 titik tangan:

- Ujung jari
- Sendi
- Pergelangan

Dari posisi titik ini, kita bisa menentukan gesture.

18.3 Kontrol Robot Menggunakan Gesture (Konsep)

Walaupun kita tidak membahas robot secara detail di sini, konsepnya adalah:

Jika gesture = open palm → berhenti

Jika gesture = fist → maju

Contoh logika sederhana:

```
if gesture == "open":  
    print("Stop")  
elif gesture == "fist":  
    print("Forward")
```

18.4 Latency dan Optimasi

Gesture recognition cukup berat.

Beberapa cara optimasi:

- Turunkan resolusi kamera
- Gunakan frame skipping
- Jangan proses setiap frame
- Gunakan threading

Contoh frame skipping:

```
if frame_count % 3 == 0:  
    process_ai(frame)
```

Refleksi Bab 18

Peserta telah memahami:

- Konsep gesture recognition
- Hand tracking dasar
- Penggunaan landmark
- Pentingnya optimasi performa

BAB 19. Intelligent Decision System

Bab ini menyatukan semua konsep sebelumnya. Sistem cerdas bukan hanya mendeteksi, tetapi mengambil keputusan berdasarkan banyak input.

19.1 Sensor Fusion (Camera + Ultrasonic)

Sensor fusion adalah penggabungan data dari beberapa sensor.

Contoh:

- Kamera mendeteksi orang
- Ultrasonik mengukur jarak

Jika orang terdeteksi DAN jarak < 50 cm $\rightarrow$ sistem berhenti

Contoh Logika Sederhana

```
if person_detected and distance < 50:  
    print("Stop")
```

19.2 Rule-based vs AI-based Decision

Rule-based

- Menggunakan if-else
- Logika ditentukan programmer
- Mudah dipahami

AI-based

- Menggunakan model
- Sistem belajar dari data
- Lebih fleksibel

Untuk level dasar, kita masih menggunakan rule-based decision.

19.3 Autonomous Behavior

Autonomous behavior berarti:

Sistem berjalan sendiri tanpa kontrol manual.

Contoh alur:

1. Kamera mendeteksi objek
2. Sensor jarak membaca lingkungan
3. Sistem memilih aksi
4. Sistem terus mengulang siklus

Inilah loop dasar sistem cerdas.

19.4 Debugging Sistem Kompleks

Semakin banyak modul, semakin kompleks sistem.

Tips debugging:

- Uji modul satu per satu
- Cetak log setiap tahap
- Cek penggunaan CPU
- Gunakan print untuk melihat state

Contoh debug sederhana:

```
print("Detected:", object_label)
print("Distance:", distance)
```

Refleksi Level Advanced

Pada Level 3 ini, peserta telah memahami:

- Object Detection
- Gesture Recognition
- Sensor Fusion
- Decision System
- Optimasi Edge AI

Penutup

Level Advanced menunjukkan bahwa Raspberry Pi mampu menjalankan:

- Computer Vision
- AI ringan
- Sistem keputusan cerdas

Semua berjalan langsung di perangkat (Edge AI).

Pada bab terakhir nanti, kita akan menyusun: **Capstone Project – Autonomous Smart System**. Sebagai rangkuman seluruh pembelajaran dari awal hingga akhir.

BAB 20 Capstone Project

Capstone Project — Eksperimen Standalone (Trainer Kit Mode)**

Pada bab ini, peserta tidak lagi hanya mempelajari konsep, tetapi **mengoperasikan sistem nyata dalam bentuk modul-modul standalone.**

Mode Standalone (TrainerKit) adalah kumpulan program independen yang dapat dijalankan satu per satu untuk mengeksplorasi kemampuan robot berbasis Computer Vision dan AI

Setiap file Python adalah:

- Aplikasi mandiri
- Tidak bergantung pada sistem utama
- Dapat dimodifikasi secara bebas
- Cocok untuk eksperimen dan riset kecil

20.1 Struktur Fitur TrainerKit

Mode Standalone menyediakan tiga kategori besar:

Computer Vision & AI

- Face Detection
- Object Detection

- Gesture Recognition
- Color Detection

Autonomous Navigation

- Line Follower
- Auto Pilot (Vision-based)
- Obstacle Avoidance

Manual Control

- Web-based controller
- Real-time video streaming

20.2 Cara Menjalankan Program

Masuk ke folder TrainerKit:

```
cd RTKAv2/TrainerKit
```

```
python3 <nama_program>.py
```

20.3 Panduan Akses: SSH, VNC, atau Web?

Agar tidak membingungkan, gunakan panduan berikut:

Gunakan SSH saja jika:

- Program hanya menampilkan log
- Tidak ada cv2.imshow
- Tidak ada web interface

Contoh:

- avoid.py
- line_follower.py (tanpa viewer)

Gunakan VNC jika:

- Program menampilkan video OpenCV
- Ada bounding box visual
- Ada visualisasi gesture atau warna

Contoh:

- face_detection.py
- object_detection.py
- gesture.py
- color_detection.py

Gunakan Web Browser jika:

- Program berbasis FastAPI
- Ada video streaming MJPEG
- Kontrol melalui WebSocket

Contoh:

- control.py

Akses melalui:

http://<IP_RASPBERRY_PI>:8000

Cek IP:

hostname -I

20.5 Program-Program Capstone

Berikut adalah program utama yang direkomendasikan sebagai Capstone Experiment.

A. Face Detection (Dasar Computer Vision)

File:

- python3 face_detection.py

Tujuan:

- Mendeteksi wajah
- Menampilkan bounding box
- Menghitung jumlah wajah
- Eksperimen:
 - Ubah confidence threshold
 - Ubah resolusi kamera
 - Uji dalam pencahayaan berbeda

B. Object Detection (AI Ringan)

File:

- python3 object_detection.py

Model:

- SSD MobileNet v2 (TFLite)
- Eksperimen:
 - Ubah confidence threshold (0.55 default)
 - Batasi hanya deteksi “person”
 - Hitung FPS sistem

C. Gesture Recognition

File:

- python3 gesture.py

Tujuan:

- Menghitung jumlah jari
- Mengenali gesture dasar
- Eksperimen:
- Ubah frame confirmation
- Uji jarak tangan
- Uji pencahayaan rendah

D. Obstacle Avoidance

File:

- python3 avoid.py

Tujuan:

- Mengukur jarak ultrasonic
- Hindari obstacle otomatis
- Eksperimen:
- Ubah safe distance (default 30cm)
- Uji reaksi di jarak 20cm
- Bandingkan respon cepat vs lambat

E. Manual Web Control

File:

- python3 control.py

Tujuan:

- Kontrol via browser
- Streaming video real-time
- Uji latency jaringan
- Eksperimen:
- Kontrol via HP
- Bandingkan WiFi 2.4GHz vs 5GHz
- Uji delay saat CPU tinggi

20.6 Konfigurasi Penting

Semua program menggunakan konfigurasi inline di dalam file.

Contoh:

Motor Pins

PIN_FL_FWD = 17

PIN_FL_BWD = 27

...

Ultrasonic

PIN_HCSR_TRIG = 26

PIN_HCSR_ECHO = 20

Camera

FRAME_WIDTH = 640

FRAME_HEIGHT = 480

*Peserta dapat mengubah parameter ini untuk eksperimen.

20.7 Eksperimen Terstruktur

- Buat laporan dengan format berikut:
- Program yang diuji
- Parameter awal
- Parameter yang diubah
- Hasil pengamatan
- Kendala
- Analisis performa
- Contoh:

Parameter	Awal	Uji	Hasil
Confidence	0.55	0.70	Lebih stabil
Resolusi	640x480	320x240	FPS naik

20.8 Troubleshooting Umum

Kamera tidak terdeteksi

```
ls /dev/video*
```

GPIO Permission Error

```
sudo usermod -a -G gpio $USER
```

Logout-login kembali.

Model AI tidak ditemukan

```
ls -l assets/
```

*Pastikan file model tersedia.

Web tidak bisa diakses

- Cek IP
- Pastikan satu jaringan
- Cek port 8000 tidak diblok

20.9 Tips Optimasi Performa Raspberry Pi

- Gunakan resolusi 320x240 untuk AI berat
- Gunakan model quantized (INT8)
- Batasi FPS
- Gunakan heatsink + fan
- Tutup aplikasi lain
- Jalankan satu program saja

20.10 Pengembangan Lanjutan

- Setelah menguasai mode standalone, peserta dapat:
- Menggabungkan face detection + avoidance

- Membuat decision system custom
- Mengembangkan model AI sendiri
- Menghubungkan ke cloud
- Membuat dashboard monitoring